

Unidad III: Servicios en la nube

Tema: Base de datos en tiempo real de Firebase (Realtime Database) con Flutter

1.Introducción conceptual

Realtime Database (RTDB) es una base de datos **JSON** alojada en la nube que sincroniza datos **en milisegundos** con todos los clientes conectados. Su fortaleza es la **baja latencia** y la **sincronización en tiempo real**; su limitación: un **modelo jerárquico** con consultas menos expresivas que Firestore (colecciones/documentos).

Cuándo preferir RTDB

- Chats, tableros, contadores, presencia, telemetría y UIs que requieren **feedback inmediato**.
- Estructuras datos **planas** y flujos con **fan-out** (escrituras a múltiples rutas).

Cuándo preferir Firestore

- Consultas compuestas, **filtros múltiples**, ordenamientos diversos, **escalabilidad multi-región** y consistencia más estricta.

2. Requisitos y configuración inicial

2.1 Prerrequisitos

- Flutter 3.x.
- Cuenta Firebase con un proyecto creado.
- Android Studio o VS Code.

2.2 Alta en Firebase y dependencias

1. Crea la app (Android/iOS/Web) en **Firebase Console**.
2. En tu proyecto Flutter, instala CLI de FlutterFire (si aún no la tienes):

```
dart pub global activate flutterfire_cli
```

```
flutterfire configure
```

Esto generará `firebase_options.dart`.

3. Añade dependencias en `pubspec.yaml`:

dependencies:

```
firebase_core: ^latest
```

```
firebase_database: ^latest
```

2.3 Inicialización en main.dart

```
import 'package:flutter/material.dart';
```

```
import 'package:firebase_core/firebase_core.dart';
```

```
import 'firebase_options.dart'; // generado por flutterfire configure
```

```
Future<void> main() async {
```

```
  WidgetsFlutterBinding.ensureInitialized();
```

```
  await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);
```

```
  // Opcional: uso del emulador local durante desarrollo
```

```
  // FirebaseDatabase.instance.useDatabaseEmulator('10.0.2.2', 9000);
```

```
  // Persistencia offline (recomendado para móviles)
```

```
  FirebaseDatabase.instance.setPersistenceEnabled(true);
```

```
  // FirebaseDatabase.instance.setPersistenceCacheSizeBytes(10 * 1024 * 1024);  
  // 10MB opcional
```

```
  runApp(const MyApp());
```

```
}
```

Nota: Si trabajas con el **Emulator Suite**, inicia: `firebase emulators:start` y habilita `useDatabaseEmulator`.

3. Modelado de datos en RTDB (JSON)

Principios

- **Desnormaliza** (evita árboles muy profundos); favorece estructuras **planas** con **fan-out** (múltiples rutas).

- Usa **keys** generadas por push() (con marca temporal incorporada) para listas.
- Añade **propiedades de orden** (p. ej., createdAt) para consultas.
- Prevé **indexOn** en reglas para cada campo que uses en consultas.

Ejemplo (chat simple):

```
{
  "messages": {
    "-Nr1...": { "text": "Hola", "uid": "u123", "createdAt": 1715700000000 },
    "-Nr2...": { "text": "¿Qué tal?", "uid": "u999", "createdAt": 1715700001500 }
  },
  "users": {
    "u123": { "name": "Ana" },
    "u999": { "name": "Luis" }
  }
}
```

4. Operaciones básicas (CRUD) y tiempo real

4.1 Referencias

```
import 'package:firebase_database/firebase_database.dart';

final db = FirebaseDatabase.instance;
final messagesRef = db.ref('messages'); // DatabaseReference
```

4.2 Crear (Create)

```
Future<void> sendMessage(String text, String uid) async {
  final newRef = messagesRef.push();
  await newRef.set({
    'text': text,
```

```

        'uid': uid,
        'createdAt': ServerValue.timestamp, // marca de tiempo de servidor
    });
}

```

4.3 Leer (Read) — instantáneo y en tiempo real

- **Una sola vez:**

```

final snap = await messagesRef.get();
if (snap.exists) {
    final data = Map<String, dynamic>.from(snap.value as Map);
}

```

Suscripción en tiempo real (onValue → Stream):

```

Stream<DatabaseEvent> stream = messagesRef.onValue;
// En UI: StreamBuilder para redibujar ante cambios

```

4.4 Actualizar (Update) y multi-path

```

Future<void> updateMessage(String id, Map<String, dynamic> patch) async {
    await messagesRef.child(id).update(patch);
}

```

// Multi-localización (fan-out):

```

Future<void> fanOutUpdate(String msgId, Map<String, dynamic> data, String uid)
async {
    final updates = {
        'messages/$msgId': data,
        'user-messages/$uid/$msgId': data,
    };
    await db.ref().update(updates);
}

```

4.5 Eliminar (Delete)

```
Future<void> deleteMessage(String id) => messagesRef.child(id).remove();
```

5. Consultas, ordenamiento y paginación

5.1 Orden y filtros

// Los hijos deben tener el campo "createdAt"; indexa en reglas:

```
final recentQuery = messagesRef
    .orderByChild('createdAt')
    .limitToLast(50);
```

```
final byUser = messagesRef
    .orderByChild('uid')
    .equalTo('u123');
```

5.2 Paginación incremental (por clave o por campo)

// Por clave (key) descendente, usa lastKey registrado del lote anterior:

```
Query pageBeforeKey(String? lastKey, int pageSize) {
    var q = messagesRef.orderByKey().limitToLast(pageSize);
    if (lastKey != null) q = q.endAt(lastKey);
    return q;
}
```

// Por campo (createdAt):

```
Query pageBeforeTs(int? lastTs, int pageSize) {
    var q = messagesRef.orderByChild('createdAt').limitToLast(pageSize);
    if (lastTs != null) q = q.endAt(lastTs * 1.0); // doble precisión requerida
    return q;
}
```

Consejo: en paginación hacia atrás con `limitToLast`, elimina el duplicado que hace de “ancla” (último elemento del lote anterior).

6. Transacciones, presencia y `onDisconnect`

6.1 Transacciones (consistencia optimista)

Útiles para contadores/stock:

```
Future<void> decrementStock(String productId) async {
  final ref = db.ref('products/$productId/stock');
  await ref.runTransaction((current) {
    final value = (current as int? ?? 0);
    if (value <= 0) return Transaction.abort(); // evita negativos
    return Transaction.success(value - 1);
  });
}
```

6.2 Presencia (online/offline) con `onDisconnect`

```
Future<void> setPresence(String uid) async {
  final statusRef = db.ref('status/$uid');
  await statusRef.onDisconnect().set({
    'state': 'offline',
    'last_changed': ServerValue.timestamp,
  });
  await statusRef.set({
    'state': 'online',
    'last_changed': ServerValue.timestamp,
  });
}
```

7. Modo offline y rendimiento

- **Persistencia:** `FirebaseDatabase.instance.setPersistenceEnabled(true);`
- **Mantener rutas sincronizadas:**

`db.ref('messages').keepSynced(true);` // útil para vistas críticas

- **Caché:** ajustar tamaño con `setPersistenceCacheSizeBytes`.
- **Evitar** `shrinkWrap` innecesario y **anidar** scrollables; usar **slivers** para composiciones complejas.
- En imágenes, usar `loadingBuilder/errorBuilder` o paquetes de caché.

8. Reglas de seguridad e índices (imprescindible)

8.1 Reglas base (autenticación requerida)

```
{
  "rules": {
    ".read": "auth != null",
    ".write": "auth != null",
    "messages": {
      ".indexOn": ["createdAt", "uid"],
      "$msgId": {
        ".read": "auth != null",
        ".write": "auth != null && newData.child('uid').val() === auth.uid",
        ".validate": "
          newData.hasChildren(['text', 'uid', 'createdAt']) &&
          newData.child('text').isString() &&
          newData.child('text').val().length <= 280 &&
          newData.child('createdAt').val() <= now
        "
      }
    }
  }
}
```

```
}  
}  
}
```

Puntos clave:

- **.indexOn** para cualquier campo de consulta (orderByChild).
- Valida tipos/longitudes y campos obligatorios.
- Autorización: el autor (uid) solo puede crear/editar sus mensajes.

9. Ejemplo integral en Flutter (UI + tiempo real)

9.1 Pantalla de mensajes con StreamBuilder (Material 3)

```
import 'package:flutter/material.dart';  
import 'package:firebase_database/firebase_database.dart';  
  
class ChatPage extends StatefulWidget {  
  const ChatPage({super.key, required this.uid});  
  final String uid;  
  
  @override  
  State<ChatPage> createState() => _ChatPageState();  
}  
  
class _ChatPageState extends State<ChatPage> {  
  final db = FirebaseDatabase.instance;  
  final _ctrl = TextEditingController();  
  
  DatabaseReference get messagesRef => db.ref('messages');  
  Query get recentQuery =>  
    messagesRef.orderByChild('createdAt').limitToLast(50);
```



```

Future<void> _send() async {
  final text = _ctrl.text.trim();
  if (text.isEmpty) return;
  _ctrl.clear();
  await messagesRef.push().set({
    'text': text,
    'uid': widget.uid,
    'createdAt': ServerValue.timestamp,
  });
}

```

@override

```

Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: const Text('Chat · Realtime DB')),
    body: Column(
      children: [
        Expanded(
          child: StreamBuilder<DatabaseEvent>(
            stream: recentQuery.onValue,
            builder: (context, snapshot) {
              if (snapshot.hasError) {
                return Center(child: Text('Error: ${snapshot.error}'));
              }
              if (!snapshot.hasData || snapshot.data!.snapshot.value == null) {
                return const Center(child: Text('Sin mensajes'));
              }
              final map = Map<String, dynamic>.from(

```

```

        snapshot.data!.snapshot.value as Map,
    );
    final list = map.entries.map((e) {
      final val = Map<String, dynamic>.from(e.value);
      return {
        'key': e.key,
        'text': val['text'] ?? "",
        'uid': val['uid'] ?? "",
        'createdAt': val['createdAt'] ?? 0,
      };
    }).toList()
    ..sort((a, b) => (a['createdAt'] as num).compareTo(b['createdAt'] as
num)));

```

```

return ListView.builder(
  padding: const EdgeInsets.all(12),
  itemCount: list.length,
  itemBuilder: (_, i) {
    final m = list[i];
    final mine = m['uid'] == widget.uid;
    return Align(
      alignment: mine ? Alignment.centerRight : Alignment.centerLeft,
      child: Card(
        color: mine
          ? Theme.of(context).colorScheme.primaryContainer
          : null,
        child: Padding(
          padding: const EdgeInsets.all(10),
          child: Text(m['text']),

```

```

        ),
      ),
    );
  },
);
},
),
),
SafeArea(
  minimum: const EdgeInsets.all(8),
  child: Row(
    children: [
      Expanded(
        child: TextField(
          controller: _ctrl,
          decoration: const InputDecoration(
            hintText: 'Escribe un mensaje',
            border: OutlineInputBorder(),
          ),
          onSubmitted: (_) => _send(),
        ),
      ),
      const SizedBox(width: 8),
      FilledButton.icon(
        onPressed: _send,
        icon: const Icon(Icons.send),
        label: const Text('Enviar'),
      ),
    ],
  ),
),

```

```

        ],
      ),
    )
  ],
),
);
}
}

```

Evidencia de competencias: stream en tiempo real, UI reactiva, ordenación por timestamp, patrón “input → set() con ServerValue.timestamp”.

10. Buenas prácticas y errores frecuentes

Buenas prácticas

- Diseña **consultas soportadas por índices** (.indexOn).
- Mantén datos **planos**; usa **fan-out** para duplicar información necesaria en diversas rutas de lectura.
- **Valida** esquemas y límites (longitud de texto, rangos, timestamps).
- Habilita **persistencia offline** y selecciona rutas con keepSynced solo si son críticas.
- Maneja **errores** con try/catch y muestra feedback (SnackBar/Dialogs).

Errores comunes

- Consultas sin índices → lecturas lentas o denegadas.
- Árboles profundamente anidados → lecturas costosas, reglas complejas.
- Usar orderByChild sobre campos no numéricos de forma inconsistente.
- Confiar en DateTime.now() del cliente (manipulable) en lugar de ServerValue.timestamp.

Glosario mínimo

- **DatabaseReference:** puntero a una ruta; se usa para set, update, remove.
- **Query:** referencia con criterios (orderBy*, limit*, startAt/endAt).
- **ServerValue.timestamp:** tiempo del servidor (evita relojes manipulados).
- **Transaction:** operación atómica de lectura-modificación-escritura.

- **onDisconnect**: acción pendiente que se ejecuta al desconectarse.

Conclusion.

Dominar **Firestore Realtime Database** en Flutter permite resolver problemas que exigen **sincronización inmediata**, tales como chats, tableros y telemetría. Al combinar **streams**, **consultas indexadas**, **transacciones** y **reglas seguras**, podrás desplegar **interfaces confiables, escalables y de baja latencia**. La clave está en **modelar bien los datos, validar con reglas y medir**: optimiza consultas e I/O para la mejor experiencia.

Ejemplo 1 — Mensajería instantánea con presencia, “escribiendo...” y paginación

1) Contexto y objetivo

Una app de chat **uno a uno** con:

- Entrega **en tiempo real** de mensajes.
- **Indicador de escritura** (“typing...”).
- **Presencia** (online/offline).
- **Paginación** (cargar historial al desplazarse).
- Seguridad por usuario.

2) Competencias

- Modelado de datos plano (fan-out).
- Streams (onValue) y StreamBuilder.
- Consultas (orderByChild, limitToLast, endAt).
- Transacciones para contadores (no leído).
- Reglas de seguridad con validación e índices.

3) Modelo de datos (JSON)

```
{
  "conversations": {
    "c_u123_u999": { "lastMessage": "Hola", "updatedAt": 1715700100000 }
  },
  "messages": {
    "c_u123_u999": {
      "-Nr1...": { "text": "Hola", "from": "u123", "to": "u999", "createdAt":
1715700000000, "read": false }
    }
  },
  "typing": {
    "c_u123_u999": { "u123": false, "u999": true }
  },
  "presence": {
    "u123": { "state": "online", "last_changed": 1715700123456 },
```

```

    "u999": { "state": "offline", "last_changed": 1715700100000 }
  }
}

```

c_u123_u999 es la **clave de conversación** (convención: c_{uidA}_{uidB} en orden lexicográfico para evitar duplicados).

4) Reglas de seguridad (extracto)

```

{
  "rules": {
    "conversations": {
      "$cid": {
        ".read": "auth != null && $cid.matches('c_' + auth.uid + '_.*') ||
$cid.matches('c_.*_' + auth.uid)",
        ".write": "auth != null",
        ".indexOn": ["updatedAt"]
      }
    },
    "messages": {
      "$cid": {
        ".read": "auth != null && ($cid.contains(auth.uid))",
        ".write": "auth != null && ($cid.contains(auth.uid))",
        "$mid": {
          ".validate": "newData.hasChildren(['text','from','to','createdAt']) &&
newData.child('from').val() == auth.uid"
        },
        ".indexOn": ["createdAt"]
      }
    },
    "typing": {
      "$cid": {

```

```

        ".read": "auth != null && ($cid.contains(auth.uid))",
        ".write": "auth != null && ($cid.contains(auth.uid))"
    }
},
"presence": {
    "$uid": {
        ".read": "auth != null && auth.uid == $uid",
        ".write": "auth != null && auth.uid == $uid"
    }
}
}
}
}
}

```

5) Código clave en Flutter

Inicialización (main.dart)

```

await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);
FirebaseDatabase.instance.setPersistenceEnabled(true);

```

Referencias básicas

```

final db = FirebaseDatabase.instance;
DatabaseReference convoRef(String cid) => db.ref('conversations/$cid');
DatabaseReference msgsRef(String cid) => db.ref('messages/$cid');
DatabaseReference typingRef(String cid) => db.ref('typing/$cid');
DatabaseReference presenceRef(String uid) => db.ref('presence/$uid');

```

Enviar mensaje (set + fan-out opcional)

```

Future<void> sendMessage(String cid, String from, String to, String text) async {
    final mref = msgsRef(cid).push();
    final payload = {
        'text': text,
        'from': from,

```



```

        'to': to,
        'createdAt': ServerValue.timestamp,
        'read': false,
    };
    await mref.set(payload);
    await convoRef(cid).update({'lastMessage': text, 'updatedAt':
ServerValue.timestamp});
}

```

Escuchar mensajes recientes (stream + paginación)

```

Query recent(String cid, {int page = 50, int? endAtTs}) {
    var q = msgsRef(cid).orderByChild('createdAt').limitToLast(page);
    if (endAtTs != null) q = q.endAt(endAtTs * 1.0);
    return q;
}

```

// UI:

```

StreamBuilder<DatabaseEvent>(
    stream: recent(cid, page: 30).onValue,
    builder: (_, snap) { /* mapear snapshot a ListView.builder */ }
);

```

Indicador “escribiendo...”

```

Future<void> setTyping(String cid, String uid, bool isTyping) async {
    await typingRef(cid).child(uid).set(isTyping);
}

```

```

StreamBuilder<DatabaseEvent>(
    stream: typingRef(cid).onValue,
    builder: (_, snap) {
        final map = (snap.data?.snapshot.value as Map?) ?? {};

```

```

    final otherTyping = (map[otherUid] as bool?) ?? false;
    return otherTyping ? const Text('Escribiendo...') : const SizedBox.shrink();
  },
);

```

Presencia con onDisconnect

```

Future<void> goOnline(String uid) async {
  final ref = presenceRef(uid);

  await ref.onDisconnect().set({'state': 'offline', 'last_changed':
ServerValue.timestamp});

  await ref.set({'state': 'online', 'last_changed': ServerValue.timestamp});
}

```

6) Rendimiento y offline

- setPersistenceEnabled(true) y **keepSynced(true)** en rutas críticas (p. ej., messages/\$cid).
- Paginación “hacia atrás”: conserva el **último timestamp** y úsalo con endAt.
- Usa ListView.builder y evita shrinkWrap salvo necesidad.

7) Extensiones

- **Leídos/entregados** con flags por mensaje.
- **Menciones y búsqueda local.**
- **Notificaciones** (Cloud Functions + FCM) al recibir mensaje.

Ejemplo 2 — Rastreo de pedidos (delivery) con ubicación en tiempo real

1) Contexto y objetivo

App de **última milla**: el repartidor publica su **geolocalización** continuamente; el cliente visualiza el **progreso en tiempo real** del pedido.

2) Competencias

- Escritura/lectura en tiempo real para **tracking**.
- onDisconnect() para estado de ruta.
- Seguridad por **pedido** y **rol** (repartidor/cliente).
- Optimización de **frecuencia de actualización**.

3) Modelo de datos

```
{
  "orders": {
    "ord_ABC": { "status": "on_route", "driverUid": "d001", "customerUid": "c777",
    "updatedAt": 1715700200000 }
  },
  "orderTracking": {
    "ord_ABC": {
      "lat": 13.6929,
      "lng": -89.2182,
      "bearing": 72.0,
      "speed": 10.2,
      "updatedAt": 1715700222000
    }
  }
}
```

4) Reglas de seguridad (extracto)

```
{
  "rules": {
    "orders": {
```

```

"$orderId": {
  ".read": "auth != null &&
(root.child('orders').child($orderId).child('customerId').val() == auth.uid ||
root.child('orders').child($orderId).child('driverUid').val() == auth.uid)",
  ".write": "auth != null && (auth.uid ==
root.child('orders').child($orderId).child('driverUid').val())",
  ".indexOn": ["updatedAt"]
}
},
"orderTracking": {
  "$orderId": {
    ".read": "auth != null &&
(root.child('orders').child($orderId).child('customerId').val() == auth.uid ||
root.child('orders').child($orderId).child('driverUid').val() == auth.uid)",
    ".write": "auth != null && (auth.uid ==
root.child('orders').child($orderId).child('driverUid').val())",
    ".validate": "newData.hasChildren(['lat','lng','updatedAt'])"
  }
}
}
}
}

```

5) Código clave en Flutter

Driver: publicar ubicación periódicamente

```

final db = FirebaseDatabase.instance;
DatabaseReference orderRef(String id) => db.ref('orders/$id');
DatabaseReference trackRef(String id) => db.ref('orderTracking/$id');

Future<void> startTracking(String orderId, String driverUid) async {
  // marca onDisconnect (opcional: status)

```

```
await orderRef(orderId).onDisconnect().update({'status': 'paused', 'updatedAt':
ServerValue.timestamp});
```

// cada X segundos/minutos (usa geolocator + Timer):

```
Future<void> publishPosition(double lat, double lng, {double? bearing, double?
speed}) async {
  await trackRef(orderId).set({
    'lat': lat, 'lng': lng, 'bearing': bearing ?? 0, 'speed': speed ?? 0,
    'updatedAt': ServerValue.timestamp,
  });
  await orderRef(orderId).update({'status': 'on_route', 'updatedAt':
ServerValue.timestamp});
}
}
```

Cliente: escuchar cambios y pintar en mapa

```
StreamBuilder<DatabaseEvent>(
  stream: trackRef(orderId).onValue,
  builder: (_, snap) {
    if (!snap.hasData || snap.data!.snapshot.value == null) return const SizedBox();
    final m = Map<String, dynamic>.from(snap.data!.snapshot.value as Map);
    final lat = (m['lat'] as num).toDouble();
    final lng = (m['lng'] as num).toDouble();
    // Integrar con google_maps_flutter o flutter_map para mover el marker
    // mapController.animateCamera(CameraUpdate.newLatLng(LatLng(lat, lng)));
    return Text('Posición: $lat, $lng ·
    ${DateTime.fromMillisecondsSinceEpoch(m['updatedAt'])}');
  },
);
```

6) Rendimiento y buenas prácticas

- **Frecuencia** de actualización ajustable (p. ej., 3–5 s) para ahorrar batería/datos.
- Combine **RTDB** con **geocodificación local** si necesita direcciones.
- Persistencia **offline**: el cliente verá la última posición cacheada.

7) Extensiones

- **ETA** (tiempo estimado) con reglas simples en el cliente o Cloud Functions.
- **Geocercas** (al llegar a destino → cambiar estado).
- **Histórico** por puntos (si requiere re-play del trayecto, guarde una lista de “pings” con push()).

Ejemplo 3 — Feed social con “likes” atómicos y contador en tiempo real

1) Contexto y objetivo

Un feed de publicaciones con:

- “**Me gusta**” instantáneos.
- **Contador** de likes en tiempo real.
- Garantía de **un solo like por usuario**.
- Timeline paginado por fecha.

2) Competencias

- Fan-out: duplicar datos mínimos para lecturas eficientes.
- **Transacciones** para contadores.
- Reglas que validan **unicidad** por usuario.
- Consultas indexadas por createdAt.

3) Modelo de datos

```
{
  "posts": {
    "p_001": {
      "authorUid": "u1",
      "text": "Hola RTDB",
      "likesCount": 3,
      "createdAt": 1715700000000
    }
  }
}
```

```

},
"postLikes": {
  "p_001": {
    "u9": true,
    "u7": true
  }
},
"userFeed": {
  "u9": { "p_001": true, "p_002": true }
}
}

```

4) Reglas de seguridad (extracto)

```

{
  "rules": {
    "posts": {
      "$pid": {
        ".read": "auth != null",
        ".write": "auth != null && auth.uid == newData.child('authorUid').val()",
        ".indexOn": ["createdAt", "likesCount"]
      }
    },
    "postLikes": {
      "$pid": {
        "$uid": {
          ".read": "auth != null",
          ".write": "auth != null && auth.uid == $uid",
          ".validate": "newData.isBoolean()" // solo true/false
        }
      }
    }
  }
}

```

```

    }
  }
}
}

```

5) Código clave en Flutter

Paginación del feed por fecha

```

final db = FirebaseDatabase.instance;
DatabaseReference postsRef() => db.ref('posts');

```

```

Query feedPage({int pageSize = 20, int? endAtTs}) {
  var q = postsRef().orderByChild('createdAt').limitToLast(pageSize);
  if (endAtTs != null) q = q.endAt(endAtTs * 1.0);
  return q;
}

```

Like atómico (toggle con transacción + fan-out)

```

Future<void> toggleLike(String pid, String uid, bool isLikedNow) async {
  final likeRef = db.ref('postLikes/$pid/$uid');
  final postRef = db.ref('posts/$pid/likesCount');

```

```

  await db.runTransaction((mutableRoot) async {
    // 1) Leer estado actual del like
    final likeSnap = await likeRef.get();
    final currentlyLiked = likeSnap.value == true;

    // 2) Calcular nuevo conteo (solo si hay cambio real)
    if (isLikedNow && !currentlyLiked) {
      await likeRef.set(true);
      await postRef.runTransaction((val) => (val as int? ?? 0) + 1);
    }
  });
}

```



```

    } else if (!isLikedNow && currentlyLiked) {
        await likeRef.remove();
        await postRef.runTransaction((val) => ((val as int? ?? 0) - 1).clamp(0, 1 <<
30));
    }
    return Transaction.success(mutableRoot);
});
}

```

UI de card con contador en vivo

```

Widget postCard(String pid) {
    final likesCountRef = db.ref('posts/$pid/likesCount');
    return StreamBuilder<DatabaseEvent>(
        stream: likesCountRef.onValue,
        builder: (_, snap) {
            final count = (snap.data?.snapshot.value as int?) ?? 0;
            return Row(
                children: [
                    IconButton(icon: const Icon(Icons.favorite_border), onPressed: () {/* llamar
toggleLike */}),
                    Text('$count'),
                ],
            );
        },
    );
}

```

6) Rendimiento y offline

- Indexa por createdAt para páginas.
- Muestra el like en UI **optimista** (feedback inmediato) y “corrige” si el stream trae un valor distinto.
- Persistencia offline: setPersistenceEnabled(true).

7) Extensiones

- **Comentarios** con paginación anidada.
- **Orden por popularidad** (combinar createdAt y likesCount).
- **Moderación** (reglas + roles de administrador).

Recomendaciones transversales (para los 3 ejemplos)

- **Persistencia offline**: habilítala globalmente; usa keepSynced(true) solo en rutas críticas.
- **Datos planos + fan-out**: duplica lo mínimo imprescindible para lecturas eficientes; mantén consistencia con multi-updates.
- **ServerValue.timestamp**: evita relojes del cliente.
- **Índices (.indexOn)**: define índices para todos los campos usados en consultas (orderByChild, orderByKey).
- **Paginación**: conserva la última clave/ts de cada lote y úsala con endAt.
- **Seguridad**: valida campos obligatorios/tipos/longitudes; amarra uid a auth.uid.

Conclusion.

Estos tres escenarios —**chat** con presencia y paginación, **rastreo de pedidos** con ubicación en tiempo real y **feed social** con “likes” atómicos— cubren patrones de **alta demanda** en apps móviles. Dominar su **modelado de datos**, **reglas de seguridad**, **streams** y **transacciones** te permitirá diseñar soluciones **rápidas, seguras y escalables** en Flutter con Firebase Realtime Database.